

Современные информационные системы немислимы без обмена данными между разнородными приложениями, работающими на разных платформах. Для решения проблемы совместимости форматов был создан расширяемый язык разметки XML (Extensible Markup Language) — метаязык, не содержащий предопределённых тегов и позволяющий разработчику самому создавать имена элементов, руководствуясь предметной областью. Эта особенность делает XML универсальным инструментом для описания практически любого класса данных: от музыкальных партитур и математических формул до баз данных и финансовых отчётов. Язык был разработан рабочей группой XML Working Group консорциума W3C в 1996 году и официально принят в 1998 году как рекомендация.

XML является упрощённой ветвью языка SGML, приспособленной для использования в Web. Наиболее известным SGML-приложением является HTML, разработанный в 1991 году. Однако HTML имеет фиксированный набор элементов и принципиально не пригоден для представления многих типов документов: в нём отсутствуют средства для отображения математических формул, музыкальных символов, иерархических структур. XML преодолевает эти ограничения, не заменяя HTML, а используя его в сочетании с ним. Согласно официальным концептуальным целям W3C, XML должен стать языком прямого использования в Internet, поддерживать большое количество приложений, быть совместимым с SGML, обладать минимальным количеством дополнительных функций, быть понятным для пользователя, а также формальным и кратким.

Описание на языке XML представляет собой операторы, написанные с соблюдением определенного синтаксиса. Когда вы создаете XML-документ, то вместо использования ограниченного набора определенных элементов вы имеете возможность создавать ваши собственные элементы и присваивать им любые имена по вашему выбору — именно поэтому язык XML является расширяемым (extensible). Следовательно, вы можете использовать XML для описания практически любого документа, от музыкальной партитуры до базы данных. Например, вы можете описать перечень книг, подобно представленному в следующем XML-документе.

Листинг 1.2. Перечень книг

```
<?xml version="1.0"?><INVENTORY>
  <BOOK><TITLE>The Adventures of Huckleberry Finn</TITLE>
    <AUTHOR>Mark Twain</AUTHOR>
    <BINDING>mass market paperback</BINDING>
    <PAGES>298</PAGES>
    <PRICE>$5.49</PRICE>
  </BOOK>
  <BOOK>
    <TITLE>Moby-Dick</TITLE>
    <AUTHOR>Herman Melville</AUTHOR>
    <BINDING>trade paperback</BINDING>
    <PAGES>605</PAGES>
    <PRICE>$4.95</PRICE>
  </BOOK>
  <BOOK>
    <TITLE>The Scarlet Letter</TITLE>
    <AUTHOR>Nathaniel Hawthorne</AUTHOR>
    <BINDING>trade paperback</BINDING>
    <PAGES>253</PAGES>
    <PRICE>$4.25</PRICE>
  </BOOK>
</INVENTORY>
```

часто используются для хранения свойств элемента, которые не являются его содержимым.

Валидность и определение типа документа

Документ, удовлетворяющий минимальным синтаксическим правилам, называется корректно сформированным. Если он дополнительно соответствует объявлению типа документа, он называется валидным. Отклонение от требований корректности считается фатальной ошибкой, тогда как нарушение валидности — лишь ошибкой. DTD описывает структуру документа: какие элементы и атрибуты допустимы, порядок их следования, типы данных и значения по умолчанию.

DTD содержит четыре основных типа инструкций. ELEMENT объявляет элементы и их содержимое, которое может быть пустым (EMPTY), любым (ANY), дочерним или смешанным. Модель содержимого может быть последовательной (элементы через запятую), выборочной (через `|`) или комбинированной. Символы `?`, `+` и `\*` управляют количеством вхождений. ATTLIST объявляет атрибуты, определяя их имена, типы (CDATA, ID, IDREF, ENTITY, NMTOKEN и другие) и значения по умолчанию (#REQUIRED, #IMPLIED, #FIXED или конкретное значение). ENTITY объявляет примитивы — часто используемые фрагменты текста или внешние файлы. NOTATION описывает формат не-XML данных.

DTD может быть внутренним, внешним или комбинированным. Внешнее подмножество размещается в отдельном файле и подключается через ключевое

Структура XML-документа

XML-документ состоит из пролога и корневого элемента. Пролог может включать объявление XML, которое указывает версию, кодировку и параметр автономности, например `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>`. Объявление не обязательно, но если оно есть, то должно находиться в самой первой строке. Пролог также может содержать объявление типа документа (DTD), инструкции по обработке и комментарии, которые начинаются с `<!--` и заканчиваются `-->`.

Корневой элемент, или элемент Документ, содержит все остальные элементы. Каждый элемент состоит из начального тега, содержимого и конечного тега. В отличие от HTML, опускать конечный тег запрещено. Имена элементов чувствительны к регистру, должны начинаться с буквы или подчёркивания и могут содержать буквы, цифры, точки, тире и подчёркивания. Префикс «xml» зарезервирован. Содержимым элемента могут быть символьные данные, вложенные элементы или их сочетание. Символы `<`, `&` и последовательность `]]>` в символьных данных недопустимы, если только они не оформлены как ссылки на символы или не помещены в раздел CDATA, который начинается с `<![CDATA[` и заканчивается `]]>`. Внутри раздела CDATA любые символы трактуются как литеральные.

Пустые элементы не имеют содержимого и записываются либо как `<HR></HR>`, либо сокращённо `<HR/>`. Они используются для указания приложению выполнить действие или для передачи информации через атрибуты. Атрибуты представляют собой пары «имя-значение», указываемые в начальном теге. Значения заключаются в одинарные или двойные кавычки.

Каждое имя атрибута должно быть уникально в пределах одного тега. Атрибуты

В рамках DTD доступны четыре определяющих инструкции для разработки определения типа документа:

- ATTLIST (список атрибутов) - объявляет список XML-атрибутов. Эти атрибуты определяются именем, типом данных, неявными значениями по умолчанию и именами любых элементов, позволяющих их использование.
- ELEMENT - объявляет имя типа XML-элемента и его допустимые вложенные (дочерние) элементы.
- ENTITY - объявляет специальные символьные ссылки, текстовые макросы (наподобие инструкции #define языка C/C++) и другое повторяющееся содержимое (наподобие инструкции #include языка C/C++).

- NOTATION - объявляет внешнее содержимое, не относящееся к XML (например, двоичные графические данные), а также внешнее приложение, которое обрабатывает это содержимое.

слово SYSTEM. Внутреннее подмножество имеет приоритет над внешним. Раздел IGNORE позволяет временно отключить блок объявлений, а замена его на INCLUDE восстанавливает их.

3.2. XML-схема (XSD)

XML-схема - это основанная на XML современная альтернатива DTD, описывающая структуру XML-документа, в том числе:

- Определяет элементы, которые могут появляться в документе;
- Определяет атрибуты, которые могут появляться в документе;
- Определяет, какие элементы являются дочерними;
- Определяет последовательность, в которой появляются дочерние элементы;
- Определяет число дочерних элементов;
- Определяет пустой ли элемент или он может включать в себя текст;
- Определяет типы данных элементов и атрибутов;
- Определяет значения атрибутов по умолчанию.

Пример XML-схемы:

```
<?xml version = "1.0" encoding = "utf-8"?>
<xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">
<!-- complexType -->
<xs:element name = "Orders">
  <xs:complexType>
    <xs:sequence>
      <xs:element name = "Person">
        <xs:complexType>
          <xs:sequence>
            <xs:element name = "Customer">
              <xs:complexType>
                <xs:sequence>
                  <xs:group ref =
"NewPersonGroup"/>
                  <xs:element name =
"Information" type = "xs:string"/>
                </xs:sequence>
              </xs:complexType>
            </xs:sequence>
          </xs:complexType>
        </xs:sequence>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:schema>
```

Листинг 7.1. Inventory01.css

```
/* File Name: Inventory01.css */
BOOK
{display:block;
margin-top:12pt;
font-size:10pt}
TITLE
{font-style:italic}
AUTHOR
{font-weight:bold}
```

Эта таблица стилей предназначена для присоединения к XML-документу, представленному в листинге 7.2, который используется и в других примерах в данной главе, поэтому вам придется не раз к нему обращаться.

Листинг 7.2. Inventory01.xml

```
<?xml version="1.0"?>
<!-- File Name: Inventory01.xml -->
<?xml-stylesheet type="text/css" href="Inventory01.css"?>
<INVENTORY>
  <BOOK>
    <TITLE>The Adventures of Huckleberry Finn</TITLE>
    <AUTHOR>Mark Twain</AUTHOR>
    <BINDING>mass market paperback</BINDING>
    <PAGES>298</PAGES>
    <PRICE>$5.49</PRICE>
  </BOOK>
  <BOOK>
    <TITLE>Leaves of Grass</TITLE>
    <AUTHOR>Walt Whitman</AUTHOR>
    <BINDING>hardcover</BINDING>
    <PAGES>462</PAGES>
    <PRICE>$7.75</PRICE>
  </BOOK>
  <BOOK>
    <TITLE>The Legend of Sleepy Hollow</TITLE>
    <AUTHOR>Washington Irving</AUTHOR>
    <BINDING>mass market paperback</BINDING>
    <PAGES>98</PAGES>
    <PRICE>$2.95</PRICE>
  </BOOK>
  <BOOK>
    <TITLE>The Marble Faun</TITLE>
    <AUTHOR>Nathaniel Hawthorne</AUTHOR>
    <BINDING>trade paperback</BINDING>
```

Современной альтернативой DTD является XML-схема (XSD), которая сама является XML-документом и поддерживает пространства имён, строгую типизацию данных и более сложные модели содержимого. XSD определяет элементы через `xs:element`, комплексные типы через `xs:complexType`, последовательности через `xs:sequence`, выборки через `xs:choice` и группы через `xs:group`.

Пространства имён, CSS и XSLT

Пространства имён позволяют избежать конфликтов имён элементов из разных словарей. Они объявляются через атрибут `xmlns`, например `xmlns:html="http://www.w3.org/TR/REC-html40/"`. Префикс действует в пределах элемента и его потомков. Это даёт возможность встраивать в XML-документы стандартные HTML-элементы.

Для отображения XML применяются CSS и XSL. CSS-таблица состоит из правил, содержащих селектор и блок объявлений. Свойства управляют шрифтом, цветом, полями, рамками, позиционированием. CSS не позволяет модифицировать структуру документа и не даёт доступа к атрибутам. XSL-таблица, напротив, содержит шаблоны преобразования, которые могут фильтровать, сортировать и реорганизовывать данные. XSLT использует XPath для навигации по документу. Шаблоны включают элементы `xsl:value-of`, `xsl:for-each`, `xsl:apply-templates`, `xsl:if`, `xsl:choose`. Атрибут `match` задаёт образец, а `select` — XPath-выражение. Фильтрация осуществляется через предикаты в квадратных скобках, сортировка — через атрибут `order-by`.

4.2. Язык преобразований XSLT

XSLT (eXtensible Stylesheet Language Transformations) - это декларативное описание преобразования (трансформации) любого XML-документа. Спецификация XSLT входит в состав XSL и является рекомендацией W3C.

Существует три основных способа преобразования XML-документов с помощью XSLT в другие форматы, например, в HTML:

- XML-документ и связанная с ним таблица стилей отправляются клиенту (веб-браузеру), который преобразует документ как указано в таблице стилей, и после этого представляет результат пользователю.
- Сервер применяет таблицу стилей XSLT к XML-документу и преобразует его в другой формат (обычно, в HTML). После этого результат отправляется клиенту (веб-браузеру).
- Какая-то программа преобразует оригинальный XML-документ в другой формат (обычно, в HTML), затем результат помещается на сервер. И сервер, и клиент имеют дело с уже преобразованным документом (рис.4.2.).

К разным документам можно применять разные таблицы стилей – и каждый раз получать различный результат. Для обращения к элементам XML-документа XSLT использует XPath. Для объявления таблицы XSLT используется корневой элемент `<xsl:stylesheet>` (возможен вариант `<xsl:transform>`, варианты равнозначны):

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

Xml-файл, содержащий таблицу XSLT, состоит из набора шаблонов. Шаблон – это набор инструкций для преобразования входного XML-документа. Для создания шаблонов используется элемент `<xsl:template>`:

```
<xsl:template match = "XPath-выражение">
  Тело шаблона
</xsl:template>
```

Структура элемента `<xsl:template>`:

```
<xsl:template
  match = pattern
```

4.1. Язык адресации XPath

XPath представляет собой язык задания путей к элементам XML-документа. Разработан для организации доступа к частям документа XML в файлах трансформации XSLT и является стандартом консорциума W3C. Для адресации частей XML-документа выражение языка XPath использует обозначение пути, похожее на обозначение в URL-адресе. Выполняется оценка выражения для того, чтобы задать объекту один из типов: тип набора узлов, логический, числовой или строковый тип. Например, выражение `book/author` возвращает набор узлов элементов

36

`<author>`, содержащихся в элементах `<book>`, при условии, что такие элементы объявлены в исходном XML-документе. Кроме того, выражение XPath может содержать предикаты (критерии фильтра) или вызовы функций. Например, выражение `book[@type="Fiction"]` ссылается на элементы `<book>`, для которых атрибут `type` принимает значение "Fiction".

как набор записей. HTML-элементы сцепляются с полями XML через атрибуты `'DATASRC'` и `'DATAFLD'`. Табличное сцепление отображает все записи в HTML-таблице, а сцепление по одной записи — по одному полю за раз. Методы `'moveFirst'`, `'moveNext'`, `'movePrevious'`, `'moveLast'` перемещают указатель между записями, а свойства `'BOF'` и `'EOF'` сигнализируют о достижении границ.

Заключение

XML прошёл путь от экспериментальной технологии до фундаментального стандарта обмена данными. Его ключевые преимущества — расширяемость, платформенная независимость, человекочитаемость, поддержка валидации и широкая экосистема инструментов — сделали его незаменимым в веб-разработке, электронной коммерции, научных исследованиях и интеграции корпоративных систем. DTD и XSD обеспечивают строгий контроль структуры, пространства имён позволяют комбинировать словари, CSS и XSLT дают гибкие возможности визуализации, DOM и XPath предоставляют программный доступ, а .NET Framework делает работу с XML удобной и эффективной.

XPath — язык адресации, описывающий пути к узлам XML-документа. Путь состоит из шагов, разделённых косой чертой. Каждый шаг включает ось (`child`, `parent`, `ancestor`, `descendant`, `attribute` и другие), проверку узла и необязательный предикат. Оси определяют направление обхода дерева. Предикаты фильтруют наборы узлов. Контекст выражения включает узел, его положение, размер контекста, привязки переменных и пространства имён.

DOM представляет документ в виде иерархического дерева узлов. Каждый узел соответствует элементу, атрибуту, тексту, комментарию или инструкции по обработке. Узлы обладают свойствами `'nodeName'`, `'nodeValue'`, `'nodeType'`, `'childNodes'`, `'firstChild'`, `'parentNode'`, `'attributes'` и другими. Доступ к элементам осуществляется через `'getElementsByTagName'`, к атрибутам — через `'getNamedItem'`. Метод `'childNodes'` возвращает объект `NodeList` с методами `'item'`, `'reset'`, `'nextNode'`.

Связывание данных позволяет HTML-странице автоматически отображать содержимое XML-элементов. Фрагмент данных `<XML ID="dso" SRC="file.xml">` создаёт объект DSO, хранящий данные